

Abstract Sytnax Tree

Ingegneria del Software Tecniche Avanzate (ISTA)



Un **Abstract Syntax Tree** (AST) è una struttura dati gerarchica a forma di albero che rappresenta la struttura sintattica di un programma. Funziona come una **rappresentazione intermedia del codice**, generata da un compilatore o da un interprete. L'AST **cattura gli elementi essenziali della sintassi del programma**, offrendo una rappresentazione organizzata di operatori, funzioni, istruzioni ed espressioni.

L'AST viene costruito attraverso il **parsing** del **codice sorgente**: durante questa fase, il compilatore analizza il codice e lo scompone nei suoi componenti fondamentali — come **keyword**, **identificatori**, **valori letterali** e **operatori**. Questi elementi vengono poi disposti in modo gerarchico sotto forma di **albero**, dove il programma principale rappresenta il nodo radice, mentre le varie parti del codice costituiscono i nodi figli.

Perché è importante un'AST?

L'AST è fondamentale perché rappresenta la **struttura logica del codice** in una forma che può essere **analizzata** e **manipolata** in modo **sicuro**, **senza bisogno di eseguire il programma**.

Attraverso l'AST, il **codice** sorgente diventa un **insieme** di **nodi** e **relazioni** che descrivono in modo preciso cosa fa il programma, indipendentemente da come è scritto.

Questo consente di realizzare strumenti come **analizzatori statici**, **formattatori**, **refactoring** automatici e controlli di sicurezza.

Intruitivamente...

Supponiamo che vogliamo identificare se tale codice presenta o meno un bug:

```
x = 12 / 0
```

Possiamo scrivere la seguente espressione regolare:

```
import re  
re.findall(r"\s*0", open("file.py").read()) //
```

Questa espressione regolare cerca nel testo ogni sequenza che contiene una slash “/” seguita da eventuali spazi e da uno zero

E se strutturassimo il codice nel seguente modo?

```
def divide(a, b):  
    return a / b
```

```
result = divide(12, 0)
```

In questo caso l'espressione regolare non sarebbe capace di identificare la divisione per zero!

Se però avessimo una struttura dati dove andassimo ad **identificare** un **nodo** corrispondente **all'operazione** di **divisione** e **due nodi figli**, uno con il divisore e uno con il dividendo, allora, verificando il valore del divisore, saremmo stati capaci di identificare la divisione per zero!

Nota bene: L'identificazione della divisione per zero in questo caso può essere fatta in modo **statico**, poiché non ci sono valori da calcolare a runtime

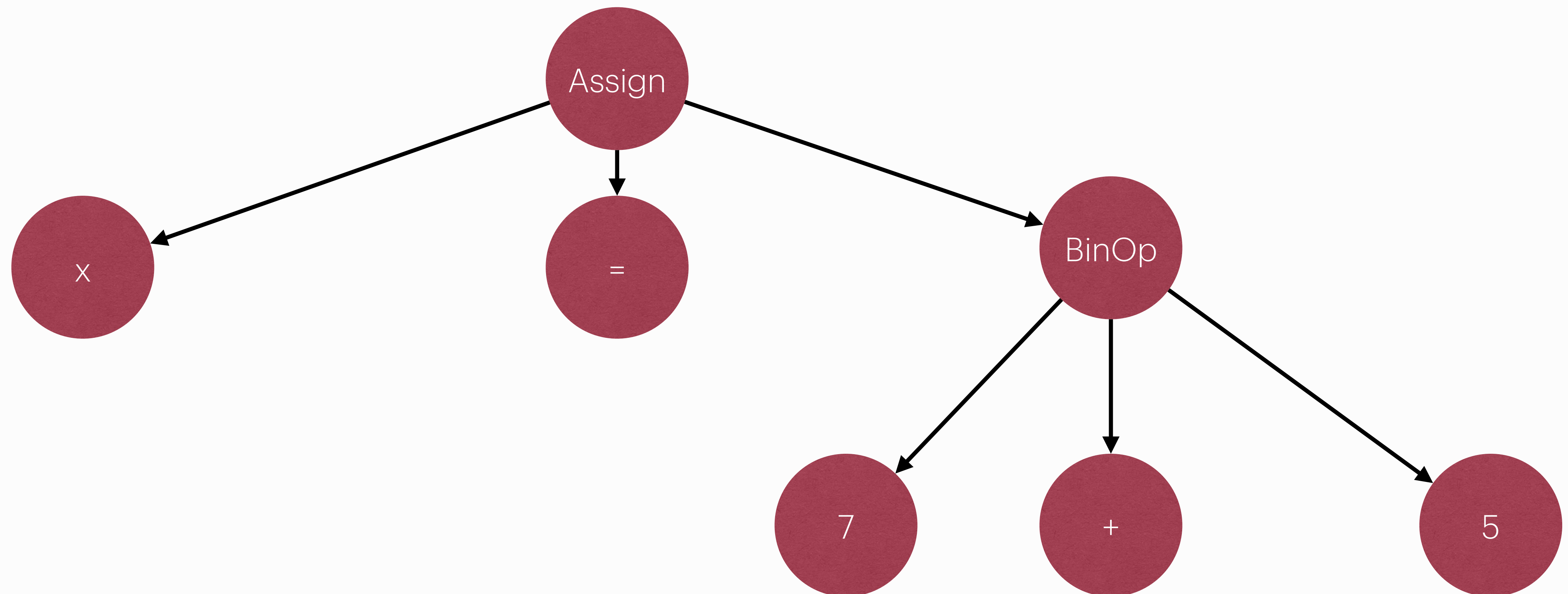
Esempi di nodi creati durante la generazione di un’AST.

Nodo AST	Categoria	Descrizione
Assign	Statement	Assegnazione di valore, es. $x = 5$. Contiene 'targets' e 'value'
BinOp	Espressione	Operazione binaria come $a + b$. Ha 'left', 'op' e 'right'
If	Struttura di controllo	Condizionale if: 'test', 'body' e 'or else'.
For	Ciclo	Ciclo for: 'target', 'iter', 'body' e 'or else'.
FunctionDef	Definizione	Definizione di funzione: 'name', 'args', 'body'.

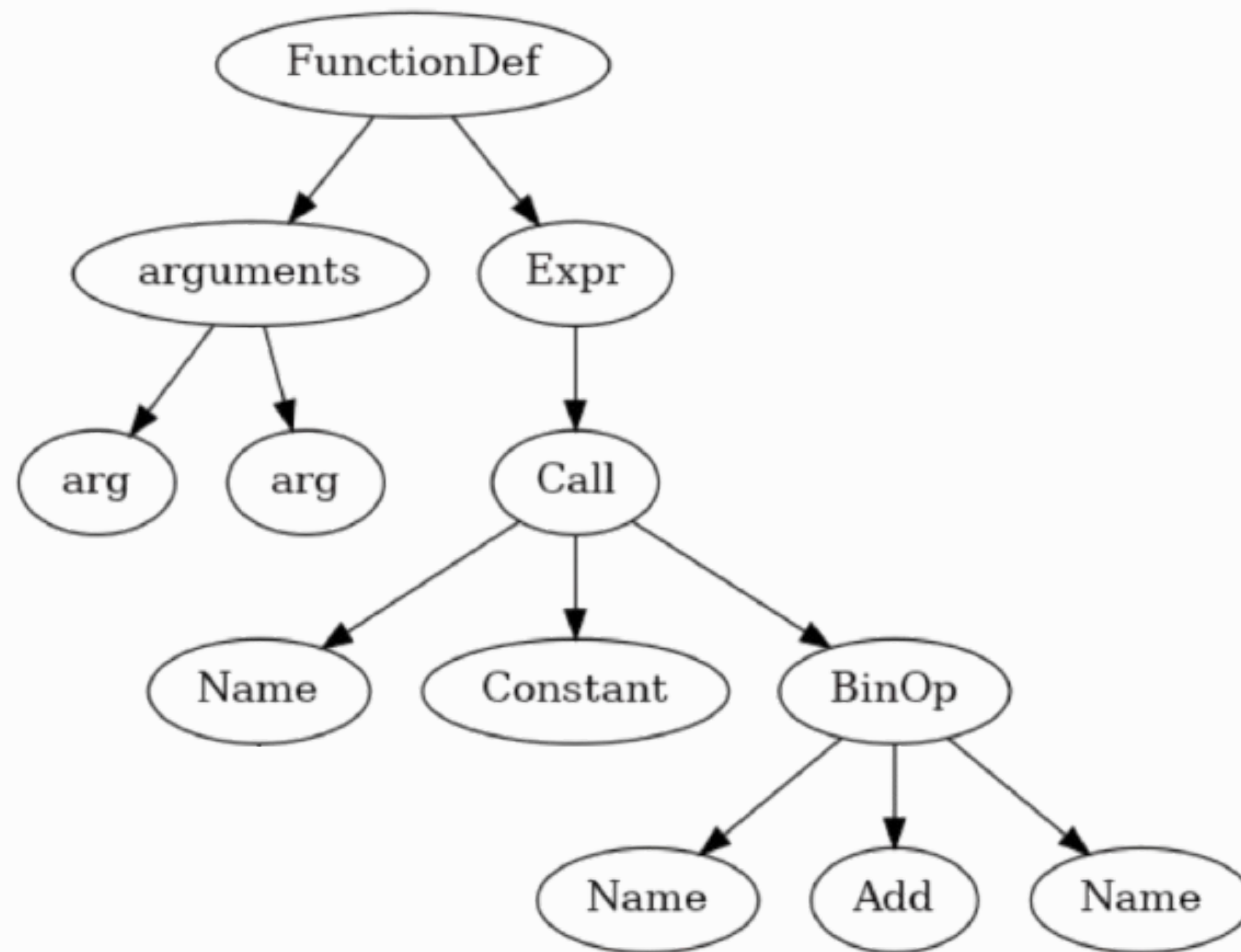
Consideriamo la seguente istruzione:

$$x = 7 + 5$$

Possiamo notare che il **lato sinistro** è un'**assegnazione**, mentre nel **lato destro** è presente un'**operazione** di addizione tra due costanti intere (7 e 5). Se considerassimo la sua rappresentazione mediante AST, otterremmo:



Un esempio più complesso potrebbe essere il seguente:



Il nodo radice è **FunctionDef**, che rappresenta la definizione di una funzione (e.g., `f()`).

Da lì partono due rami principali:

arguments, che contiene gli argomenti della funzione (arg, arg).

Expr, che rappresenta l'unica espressione nel corpo della funzione.

Sotto **Expr** c'è:

Call → una chiamata di funzione

Il **primo figlio Name** (e.g., `.print(...)`).

Il **secondo figlio Constant** (e.g. 1)

Il **terzo figlio** è un **BinOp** (operazione binaria), con:

Name → (e.g., `a`),

Add (l'operatore `+`),

Name → Load (e.g., `b`).

Una semplice preliminare implementazione

Python offre di default una libreria che può essere utilizzata per creare un'AST

```
import ast
```

Alcuni metodi della libreria:

Descrizione	Descrizione
ast.parse(source)	Converte una stringa di codice Python in un albero sintattico . È il punto di partenza di qualsiasi analisi.
ast.dump(node, indent=4)	Mostra l'AST in forma leggibile. Perfetto per capire la struttura del codice.
ast.walk(node)	Permette di attraversare tutti i nodi dell'albero , uno per uno. Utile per cercare un certo tipo di nodo (es. tutte le chiamate a funzione).
ast.NodeVisitor	Classe base per creare un <i>visitor</i> : definisci metodi visit_<Nodo> per analizzare solo i nodi che ti interessano.
ast.unparse(node)	Ricostruisce il codice Python originale (o modificato) a partire da un AST.

Una semplice preliminare implementazione — Stampa dell'oggetto

Consideriamo il seguente snippet di codice:

Main.py

```
import ast
code = "x = a + b"
tree = ast.parse(code) #lo converte in oggetto Python che rappresenta la struttura del programma
print(tree)
```

Output

```
<_ast.Module object at 0x7f987f9c35b0>
```

```
** Process exited – Return Code: 0 **
```

Una semplice preliminare implementazione — Mostra l'AST

Consideriamo il seguente snippet di codice:

Main.py

```
import ast
code = "x = a + b"
tree = ast.parse(code) #lo converte in oggetto Python che rappresenta la struttura del programma
print(ast.dump(tree))
```

Output

```
Module(body=[Assign(targets=[Name(id='x', ctx=Store())], value=BinOp(left=Name(id='a', ctx=Load()), op=Add(), right=Name(id='b', ctx=Load())), type_comment=None)], type_ignores=[])
```

```
** Process exited - Return Code: 0 **
```

Una semplice preliminare implementazione — Attraversa i nodi

Consideriamo il seguente snippet di codice:

Main.py

```
import ast
code = "x = a + b"
tree = ast.parse(code) #lo converte in oggetto Python che rappresenta la struttura del programma
for node in ast.walk(tree):
    print(type(node))
```

Output

```
<class '_ast.Name'>
<class '_ast.Add'>
<class '_ast.Name'>
<class '_ast.Load'>
<class '_ast.Load'>
```

Una semplice preliminare implementazione — Accesso diretto ai nodi

Consideriamo il seguente snippet di codice:

Main.py

```
assign_node = tree.body[0]
print(assign_node.targets[0].id)    # 'x'
print(assign_node.value.left.id)    # 'a'
print(assign_node.value.op)         # <_ast.Add object>
print(assign_node.value.right.id)   # 'b'
```

Per ulteriori approfondimenti fate riferimento alla documentazione: <https://docs.python.org/3/library/ast.html>

Abbiamo l'AST. Ora: come facciamo ad analizzarlo in modo sistematico?

Visitor Pattern. Il Visitor Pattern è un modo ordinato per **attraversare un albero sintattico** e analizzarne solo le parti che ci interessano. Invece di scrivere codice che esplora manualmente tutti i nodi, il visitor si occupa automaticamente di “camminare” nell'AST e di richiamare, per ogni tipo di nodo, un metodo dedicato. Questo rende l'analisi più pulita, modulare e facile da estendere: se vogliamo fare qualcosa di diverso su un nodo Assign o su un nodo If, ci basta definire un metodo visit_Assign o visit_If. Il visitor è quindi lo strumento che trasforma l'AST da semplice struttura dati in un meccanismo attivo di analisi e comprensione del codice.

Come funziona (idea chiave):

Erediti da ast.NodeVisitor.

Definisci solo i metodi dei nodi che ti interessano.

visit(tree) fa dispatch dinamico: se esiste visit_Call lo usa, altrimenti richiama generic_visit.

Visitor Pattern — Esempio: Raccogliere nomi di funzioni chiamate

```
import ast

class CallCollector(ast.NodeVisitor):
    # Visitor che raccoglie i nomi delle funzioni chiamate
    def __init__(self):
        # Costruttore
        self.calls = set()
        # Insieme per evitare duplicati (es. {'print', 'append'})

    def visit_Call(self, node):
        # Chiamato automaticamente per ogni nodo di tipo Call
        # chiamate tipo: foo(...), print(...)
        if isinstance(node.func, ast.Name):
            # Se la funzione è un nome semplice
            self.calls.add(node.func.id)
            # Aggiunge l'identificatore (es. 'print')
        # chiamate tipo: obj.metodo(...)
        elif isinstance(node.func, ast.Attribute):
            # Se è un attributo di un oggetto
            self.calls.add(node.func.attr)
            # Aggiunge il nome del metodo (es. 'append')
        self.generic_visit(node)
        # Continua la visita su argomenti e sotto-nodi

def main():
    source = """
def f(a, b):
    x = a + b
    print(x)
"""
    tree = ast.parse(source) # AST del codice
    v = CallCollector()      # Istanza il visitor
    v.visit(tree)           # Visita l'AST 'tree'
    print("Funzioni chiamate:", v.calls) # Stampa l'insieme raccolto (es. {'print'})
```

Visitor Pattern — Esempio: Modifica di una informazione

```
import ast

# Codice sorgente originale
source = "x = 1 + 2"

# Parsing del codice in un AST
tree = ast.parse(source)

# Definizione di un NodeTransformer personalizzato
class ReplaceOneWithTwelve(ast.NodeTransformer):
    def visit_Constant(self, node):
        # Se il nodo è il numero 1, lo sostituiamo con 12
        if isinstance(node.value, int) and node.value == 1:
            new_node = ast.Constant(value=12)
            return ast.copy_location(new_node, node)
        return node

# Applica la trasformazione
transformer = ReplaceOneWithTwelve()
new_tree = transformer.visit(tree)

# Corregge eventuali metadati interni dopo la modifica
ast.fix_missing_locations(new_tree)

# Compila il nuovo AST in codice eseguibile
code = compile(new_tree, filename="<ast>", mode="exec")

# Esegui il codice modificato
exec(code)

# Controlla il risultato
print(x) # Stampa: 14 (perché 12 + 2 = 14)
```

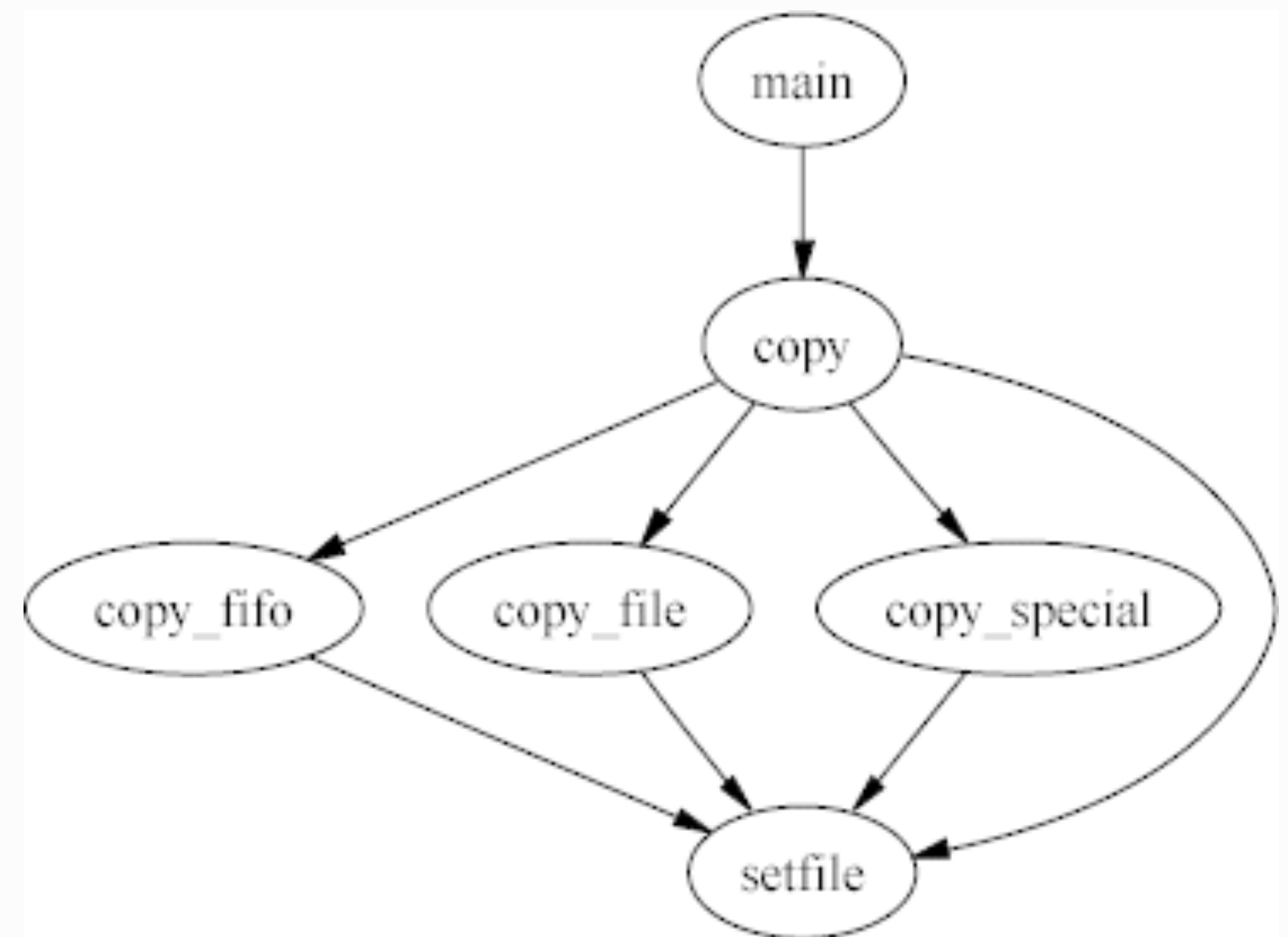
Modifica del valore del nodo. Da 1 passa a 12

Visitor Pattern — Esempio: Mini Call Graph

Call Graph. Un Call Graph è una rappresentazione del programma che mostra **quali funzioni chiamano quali altre funzioni**. Ogni **nodo** del grafo **rappresenta** una **funzione**, e ogni **freccia** (arco) **rappresenta** una **chiamata** tra funzioni. In pratica, è una mappa delle relazioni tra le parti del codice: ci permette di capire come il flusso dell'esecuzione si muove da una funzione all'altra.

Questo tipo di analisi è molto utile per:

- comprendere programmi complessi;
- Trovare dipendenze o cicli di chiamate;
- Eseguire refactoring in modo sicuro;
- Individuare funzioni inutilizzate o troppo dipendenti da altre.



Visitor Pattern — Esempio: Mini Call Graph

```
import ast
from collections import defaultdict

class CallGraphVisitor(ast.NodeVisitor):
    def __init__(self):
        self.current_fn = None
        self.graph = defaultdict(set)

    def visit_FunctionDef(self, node):
        prev = self.current_fn
        self.current_fn = node.name
        self.generic_visit(node)
        self.current_fn = prev

    def visit_Call(self, node):
        if self.current_fn:
            if isinstance(node.func, ast.Name):
                self.graph[self.current_fn].add(node.func.id)
            elif isinstance(node.func, ast.Attribute):
                self.graph[self.current_fn].add(node.func.attr)
            self.generic_visit(node)

cg = CallGraphVisitor()
cg.visit(tree)
print(dict(cg.graph))
```

Importiamo defaultdict per creare facilmente dizionari di insiemi

Definiamo una classe Visitor personalizzata per costruire un call graph

Tiene traccia del nome della funzione attualmente visitata

Dizionario: {funzione_chiamante -> {funzioni_chiamate}}

Metodo chiamato quando troviamo una definizione di funzione (FunctionDef)

Salviamo la funzione corrente (nel caso di funzioni annidate)

Aggiorniamo la funzione corrente con il nome della funzione trovata

Visitiamo ricorsivamente tutto il corpo della funzione

Ripristiniamo la funzione precedente una volta finita la visita

Metodo chiamato per ogni chiamata di funzione trovata (Call)

Solo se siamo dentro una funzione (cioè c'è un chiamante)

Caso: chiamata semplice come print(x) o foo(a)

Aggiunge la relazione caller -> callee

Caso: chiamata a metodo tipo obj.metodo()

Aggiunge solo il nome del metodo (callee)

Continua a visitare eventuali sotto-nodi (argomenti ecc.)

Istanza il nostro visitor

Avvia la visita sull'AST 'tree'

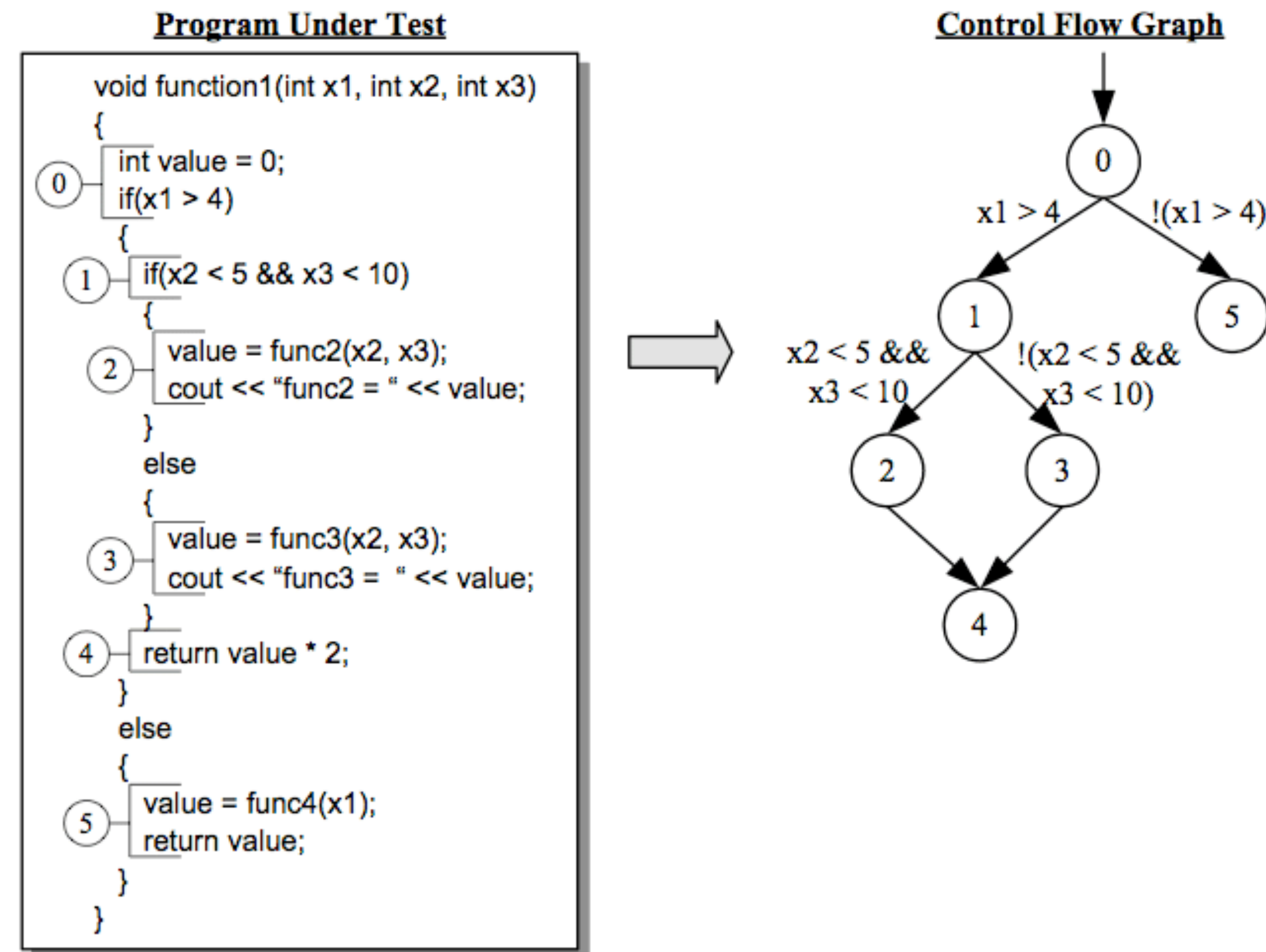
Mostra il grafo come dizionario, es: {'f': {'print'}}

Visitor Pattern — Esempio: Control Flow Graph

Control Flow Graph (CFG). Un **CFG** rappresenta il **flusso di esecuzione** di un programma.

I **nodi** sono istruzioni o blocchi (p.es. un if, un'assegnazione, un while), e gli **archi** indicano **dove può proseguire l'esecuzione**: sequenza lineare, biforcazioni (if), cicli (while), salti a fine blocco, ecc.

Serve per capire percorsi possibili, copertura dei test, individuare codice morto e ragionare su proprietà come “posso arrivare qui?” o “quali cammini esistono?”.



Visitor Pattern — Esempio: Control Flow Graph

In pratica, prende il codice della funzione e prova a “disegnare” come scorre l’esecuzione — cioè da dove parte, dove può andare e dove finisce.

1. Prima legge il testo della funzione e lo trasforma in un AST, cioè la struttura logica del programma.
2. Poi scorre tutte le istruzioni della funzione, una per una.
3. Se trova un’istruzione normale (come un’assegnazione o un return), la collega in sequenza alla precedente. → ad esempio: ENTRY → $a = 1$ → $b = a * 2$ → return b
4. Se trova un if, crea tre blocchi:
 1. uno per il test (IF $x > 0$),
 2. uno per il ramo “then”,
 3. uno per il ramo “else”,

Poi un punto di riunione (AFTER_IF) dove i due rami si ricongiungono.

In pratica, disegna i due percorsi alternativi dell’if.

5. Alla fine aggiunge un nodo END per indicare l’uscita dal programma.

Visitor Pattern — Esempio: Control Flow Graph

```
import ast                                # Modulo standard per lavorare con l'AST

code = """                                # Sorgente di esempio da analizzare
def f(x):
    a = 1
    if x > 0:
        a = a + 1
    b = a * 2
    return b
"""

tree = ast.parse(code)                   # Converti il testo in un albero AST
func = next(n for n in tree.body         # Prende la prima definizione di funzione nel modulo
            if isinstance(n, ast.FunctionDef))

edges = []                               # Qui raccoglieremo gli archi del CFG come tuple (da -> a)
prev = "ENTRY"                           # Nodo fittizio di ingresso al grafo

for stmt in func.body:                   # Itera sugli statement del corpo della funzione, in ordine
    if isinstance(stmt, ast.If):          # Se lo statement è un 'if', apriamo la biforcazione
        test = f"IF {ast.unparse(stmt.test)}" # Etichetta del nodo di test (es. "IF x > 0")
        edges.append((prev, test))        # Arco dal nodo precedente al test dell'if

        if stmt.body:                     # Ramo THEN: se c'è almeno uno statement nel then
            then_lbl = f"THEN {ast.unparse(stmt.body[0])}" # Etichetta con la prima istruzione del then
            edges.append((test, then_lbl)) # Arco test -> primo nodo del then
        else:                              # Se il then è vuoto, usiamo un segnaposto (nodo fittizio)
            then_lbl = "THEN pass"

        if stmt.orelse:                     # Ramo ELSE: se esiste almeno una istruzione nell'else
            else_lbl = f"ELSE {ast.unparse(stmt.orelse[0])}" # Etichetta con la prima istruzione dell'else
            edges.append((test, else_lbl)) # Arco test -> primo nodo dell'else
        else:                              # Se l'else non c'è, nodo fittizio per rappresentare il ramo
            else_lbl = "ELSE (vuoto)"
            edges.append((test, else_lbl)) # Arco test -> segnaposto dell'else
```

Visitor Pattern — Esempio: Control Flow Graph

```
after = "AFTER_IF"           # Nodo di riunione (join) dopo l'if
edges.append((then_lbl, after)) # Arco dal ramo THEN verso il join
if else_lbl != "ELSE (vuoto)": # Se l'else aveva una vera istruzione
    edges.append((else_lbl, after)) # Arco dal ramo ELSE verso il join
else:
    edges.append((else_lbl, after)) # Anche il segnaposto converge nel join (per simmetria)
prev = after                  # Dopo l'if, il "precedente" diventa il nodo di join

else:
    label = ast.unparse(stmt)    # Caso generale: statement lineare (Assign, Return, ecc.)
    edges.append((prev, label))  # Etichetta leggibile dello statement (es. "a = 1")
    prev = label                # Aggiungi arco dal precedente a questo statement
                                # Questo statement diventa il nuovo "precedente"

edges.append((prev, "END"))      # Alla fine, collega l'ultimo nodo a un nodo di uscita "END"

for e in edges:                  # Stampa tutti gli archi del CFG
    print(e)
```

Esempi di tool che usano AST

Molti strumenti Python di analisi del codice, formattazione, sicurezza o type checking **non eseguono** il codice — lo **analizzano staticamente**.

Per farlo, leggono il sorgente, lo trasformano in un **AST** (Abstract Syntax Tree) e poi lo ispezionano.

Così possono:

- trovare bug, violazioni di stile o vulnerabilità;
- comprendere la struttura logica del programma;
- modificare o rigenerare codice automaticamente.

Esempi di tool che usano AST

flake8

Fa' analisi statica del codice per controllare lo stile (PEP8), errori logici e pattern sospetti.

Usa AST per capire la **struttura del codice** e segnalare problemi come:

- variabili non usate;
- funzioni troppo lunghe;
- import inutilizzati.

black

È un **formatter automatico** (code formatter).

Usa AST per “capire” il codice *indipendentemente dalla formattazione originaria*:

- prima lo **parsa in un AST**;
- poi **rigenera** il codice con le proprie regole di stile.

Abstract Sytnax Tree

Ingegneria del Software Tecniche Avanzate (ISTA)

